

Clase 4: Introducción a NumPy y Análisis Exploratorio de Datos (EDA) Básico

Asignatura: Inteligencia Artificial

Clave: InteligenciaArtificialN

Duración: 90 minutos

Semana: 4

Repaso de la Clase Anterior (3 min)

- **Manejo de archivos:**
 - Aprendimos a leer y escribir archivos de texto, CSV y JSON.
 - Usamos `csv.DictReader` para convertir datos tabulares en una lista de diccionarios de Python.
 - Aprendimos a manejar errores con `try-except` para que nuestros programas sean más robustos.
- **Estructuras de datos anidadas:**
 - Creamos listas de diccionarios para representar datos estructurados (ej. cultivos).
- **Conexión con el proyecto:**
 - El manejo de archivos es el primer paso para trabajar con datos reales. Ahora que sabemos cargarlos, el siguiente paso es analizarlos de forma eficiente. Y para eso, necesitamos NumPy.

Agenda de la Clase

1. Explicación:

- ¿Qué es NumPy y por qué es la base de la IA en Python?
- Creación de arrays: la diferencia fundamental con las listas.
- Operaciones vectorizadas: la clave de la eficiencia.
- Indexación, slicing y broadcasting: el poder de manipular datos.
- Introducción al Análisis Exploratorio de Datos (EDA) con NumPy y Matplotlib.

2. Actividad Práctica en Clase:

- Cargar datos desde un archivo CSV.
- Explorar y analizar los datos usando NumPy.
- Generar visualizaciones básicas con Matplotlib para responder preguntas sobre los datos.

3. Actividad Independiente:

- Actividad para la siguiente sesión, enfocada en el avance del proyecto.

¿Qué es NumPy y por qué es la Base de la IA en Python? (5 min)

NumPy (Numerical Python) es la biblioteca fundamental para el cómputo científico en Python. Es la base sobre la que se construyen todas las demás bibliotecas de ciencia de datos e IA, como Pandas, Scikit-learn, TensorFlow y PyTorch.

¿Por qué es tan importante?

Imaginar que se necesita sumar 1 a cada uno de los 10 millones de números en una lista. Con una lista de Python, se tendría que usar un bucle for, lo cual sería lento. NumPy **vectoriza** estas operaciones, lo que significa que la operación se aplica a todo el array de una vez, en código C optimizado, haciéndolo **mucho más rápido**.

Analogía: se puede pensar en una lista de Python como una fila de personas en una tienda, donde cada persona debe ser atendida una por una. Un array de NumPy es como una cinta transportadora que mueve a todos a la vez, realizando la operación (como sumar 1) de forma simultánea y eficiente.

En resumen: NumPy es la herramienta que se necesita para manejar grandes volúmenes de datos de manera eficiente y es el pilar de cualquier proyecto de IA en Python.

Configuración del Entorno (Dentro de WSL y Docker) (5 min)

Antes de empezar, se debe asegurar de que todos tienen el entorno listo. Como se trabaja con WSL y Docker, es importante recordar los pasos para ejecutar el código.

¿Qué es un entorno virtual (venv)?

Un entorno virtual es un directorio aislado que contiene su propia instalación de Python y sus propias bibliotecas. Sirve para:

- **Aislar dependencias:** Cada proyecto puede tener sus propias versiones de bibliotecas sin afectar a otros proyectos ni al sistema operativo.
- **Evitar conflictos:** Si dos proyectos requieren versiones diferentes de una misma biblioteca, los entornos virtuales evitan que choquen.
- **Reproducibilidad:** Al tener un requirements.txt, podemos recrear el mismo entorno en cualquier máquina.

¿Dónde se debe ejecutar source venv/bin/activate?

La respuesta depende de **dónde** se esté trabajando:

- **Dentro del contenedor Docker:** se debe configurar el Dockerfile para crear un entorno virtual en /opt/venv y configura el PATH para que ese entorno sea el predeterminado. Por lo tanto, **es necesario activarlo** dentro del contenedor. Después, al ejecutar python o pip dentro del contenedor, ya se usará el entorno virtual de /opt/venv.
- **En el host (WSL), fuera del contenedor:** Si se quiere ejecutar código directamente en WSL (sin usar Docker), entonces deben crear su propio entorno virtual (por ejemplo, en ~/proyecto/venv) y activarlo con source venv/bin/activate.

Actualizar Dockerfile

```
FROM python:3.12-slim
WORKDIR /app

# 1. Crear el entorno virtual en una ruta limpia
RUN python -m venv /opt/venv

# 2. Activar el entorno virtual agregándolo al PATH
ENV PATH="/opt/venv/bin:$PATH"

COPY requirements.txt .

# 3. Este comando ya usará el pip del venv de forma automática
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/
CMD ["python", "src/main.py"]
```

¿Por qué funciona así?

Al poner `/opt/venv/bin` al principio de la variable ENV PATH, se le está indicando al contenedor que, cada vez que se ejecute `pip` o `python`, busque **primero** dentro del entorno virtual. Esto simula una activación permanente sin necesidad de escribir `source venv/bin/activate` en cada paso.

Después es necesario reconstruir el contenedor para que tome los cambios:

```
docker compose build
```

¿Cómo deben trabajar en este curso?

Dado que el proyecto está configurado con Docker y docker-compose, la forma recomendada y más sencilla es **trabajar dentro del contenedor**. Esto garantiza que todos tengan exactamente el mismo entorno y que las bibliotecas (NumPy, Matplotlib, etc.) ya estén instaladas según el requirements.txt.

Flujo de trabajo recomendado

Navegar a la carpeta del proyecto (donde está el docker-compose.yml):

```
cd ~/proyectos/ ia-python
```

Levantar el contenedor (si no está corriendo):

```
docker-compose up -d
```

Abrir una terminal dentro del contenedor:

```
docker exec -it python_ia bash
```

Nota: python_ia es el nombre del contenedor definido en el docker-compose.yml.

Dentro del contenedor, ya están en el entorno virtual /opt/venv. Pueden verificar con:

```
which python
```

Debe mostrar: /opt/venv/bin/Python

Ejecutar scripts dentro del contenedor:

```
python eda_cultivos.py
```

Si se necesita instalar una nueva biblioteca (aunque no debería ser necesario porque está en requirements.txt), pueden ejecutar:

```
pip install nombre_biblioteca
```

Y deben recordar actualizar el requirements.txt para que persista en el contenedor.

Creación de Arrays: La Diferencia Fundamental con las Listas

Importar NumPy

La convención estándar es importar NumPy como np.

```
import numpy as np
```

Crear un Array desde una Lista (crear archivo array_lista.py)

La forma más básica de crear un array es a partir de una lista de Python.

```
# Crear una lista de Python
lista_python = [1, 2, 3, 4, 5]

# Crear un array de NumPy a partir de la lista
array_numpy = np.array(lista_python)

print("Lista de Python:", lista_python)
print("Array de NumPy:", array_numpy)
print("Tipo de array:", type(array_numpy))
```

Salida:

```
Lista de Python: [1, 2, 3, 4, 5]
Array de NumPy: [1 2 3 4 5]
Tipo de array: <class 'numpy.ndarray'>
```

Arrays Multidimensionales (crear array_multidimensionales.py)

NumPy puede manejar arrays de múltiples dimensiones (matrices).

```
# Crear un array 2D (matriz) a partir de una lista de listas
matriz = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print("Array 2D:\n", matriz)
print("Dimensiones (shape):", matriz.shape) # (filas, columnas)
```

Salida:

```
Array 2D:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Dimensiones (shape): (3, 3)
```

Explicación:

- .shape es un atributo muy importante. Indica las dimensiones del array. En este caso, es una matriz de 3x3.

Funciones Útiles para Crear Arrays

NumPy ofrece funciones para crear arrays sin necesidad de listas predefinidas

Función	Descripción	Ejemplo
np.zeros()	Crea un array lleno de ceros.	np.zeros((2, 3)) → array de 2x3 con ceros.
np.ones()	Crea un array lleno de unos.	np.ones((3, 2)) → array de 3x2 con unos.
np.arange()	Crea un array con una secuencia de números, similar a range().	np.arange(0, 10, 2) → [0, 2, 4, 6, 8].
np.linspace()	Crea un array con un número específico de puntos entre un inicio y un fin.	np.linspace(0, 1, 5) → [0., 0.25, 0.5, 0.75, 1.].
np.random.rand()	Crea un array de números aleatorios entre 0 y 1.	np.random.rand(2, 3) → array de 2x3 con números aleatorios.

Ejemplo: (funciones_utiles.py)

```
# Array de ceros de 2 filas y 3 columnas
ceros = np.zeros((2, 3))
print("Ceros:\n", ceros)

# Array de números del 0 al 9
secuencia = np.arange(10)
print("Secuencia:", secuencia)

# 5 números igualmente espaciados entre 0 y 1
espaciados = np.linspace(0, 1, 5)
print("Espaciados:", espaciados)
```

Operaciones Vectorizadas: La Clave de la Eficiencia (8 min)

Esta es la funcionalidad más importante de NumPy. Las operaciones se aplican elemento por elemento de forma automática y eficiente.

Operaciones Aritméticas Básicas (operaciones.py)

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

# Suma elemento a elemento
print("Suma:", a + b)      # [11 22 33 44]

# Resta
print("Resta:", b - a)     # [ 9 18 27 36]

# Multiplicación
print("Multiplicación:", a * b) # [ 10  40  90 160]

# División
print("División:", b / a)   # [10. 10. 10. 10.]

# Potencia
print("Potencia:", a ** 2)  # [ 1  4  9 16]
```

Operaciones con un Escalar (escalar.py)

Operar un array con un solo número (escalar¹) también es vectorizado.

```
c = np.array([1, 2, 3, 4])

# Sumar 10 a cada elemento
print(c + 10) # [11 12 13 14]

# Multiplicar cada elemento por 2
print(c * 2)  # [ 2  4  6  8]
```

¹ Un escalar es una cantidad o un número que solo tiene magnitud (un valor numérico y su unidad), sin dirección ni sentido

Comparación con Listas de Python (Demostración de Rendimiento)

Se puede hacer una pequeña demostración para mostrar la diferencia de velocidad (rendimiento.py).

```
import numpy as np
import time

# Crear una lista y un array muy grandes
tamano = 1_000_000
lista_grande = list(range(tamano))
array_grande = np.arange(tamano)

# Medir tiempo con lista de Python
inicio = time.time()
lista_resultado = [x * 2 for x in lista_grande]
fin = time.time()
print(f"Tiempo con lista de Python: {fin - inicio:.4f} segundos")

# Medir tiempo con array de NumPy
inicio = time.time()
array_resultado = array_grande * 2
fin = time.time()
print(f"Tiempo con array de NumPy: {fin - inicio:.4f} segundos")
```

Salida esperada: El tiempo con NumPy será significativamente menor (del orden de 10 a 100 veces más rápido).

Conclusión: Para operaciones numéricas a gran escala, NumPy es la opción correcta.

Indexación, Slicing y Broadcasting: El Poder de Manipular Datos

Indexación y Slicing (Rebanado)

Es muy similar a las listas de Python, pero se extiende a múltiples dimensiones.
(slicing.py)

```
import numpy as np

# Array 1D
arr = np.array([10, 20, 30, 40, 50, 60, 70])

# Indexación
print(arr[0])    # 10
print(arr[-1])   # 70

# Slicing
print(arr[1:4])   # [20 30 40] (elementos del índice 1 al 3)
print(arr[:3])    # [10 20 30] (primeros 3)
print(arr[3:])    # [40 50 60 70] (desde el índice 3)
print(arr[::2])   # [10 30 50 70] (de 2 en 2)

# Array 2D (matriz)
matriz = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])

# Indexación: fila 1, columna 2
print(matriz[1, 2]) # 7

# Slicing: primeras 2 filas, columnas 1 y 2
print(matriz[:2, 1:3])
# [[2 3]
#  [6 7]]
```

Broadcasting

El *broadcasting* permite que NumPy realice operaciones entre arrays de diferentes formas sin necesidad de copiar datos, lo que ahorra memoria y tiempo.

Regla: NumPy compara las dimensiones de los arrays de atrás hacia adelante. Son compatibles si:

1. Son iguales.
2. Una de ellas es 1.

```
import numpy as np

# Ejemplo 1: Array 2D + Array 1D
a = np.array([[1, 2, 3], [4, 5, 6]]) # Shape (2, 3)
b = np.array([10, 20, 30])          # Shape (3,)

# b se "estira" para coincidir con a
resultado = a + b
print(resultado)
# [[11 22 33]
#  [14 25 36]]

# Ejemplo 2: Array 2D + Escalar
escalar = 5
print(a * escalar)
# [[ 5 10 15]
#  [20 25 30]]
```

Introducción al Análisis Exploratorio de Datos (EDA) con NumPy y Matplotlib

El **Análisis Exploratorio de Datos (EDA)** es el proceso de investigar un conjunto de datos para resumir sus características principales, a menudo con métodos visuales. Es un paso **obligatorio** antes de construir cualquier modelo de IA.

Objetivos del EDA:

- Entender la estructura de los datos (¿cuántas filas y columnas?).
- Identificar valores faltantes o atípicos.
- Encontrar relaciones entre variables.
- Generar hipótesis para el modelado.

Estadísticas Descriptivas con NumPy

NumPy proporciona funciones para calcular estadísticas básicas de forma rápida. (estadisticas_desc.py)

```
import numpy as np

# Crear datos de ejemplo (temperaturas en Cartago)
temperaturas = np.array([28, 30, 25, 27, 29, 31, 26, 24, 32, 28])

# Estadísticas básicas
media = np.mean(temperaturas)      # Promedio
mediana = np.median(temperaturas)  # Mediana
desviacion = np.std(temperaturas)  # Desviación estándar
maximo = np.max(temperaturas)      # Valor máximo
minimo = np.min(temperaturas)      # Valor mínimo

print(f"Media: {media:.2f}")
print(f"Mediana: {mediana:.2f}")
print(f"Desviación estándar: {desviacion:.2f}")
print(f"Máximo: {maximo}")
print(f"Mínimo: {minimo}")
```

Visualización Básica con Matplotlib

Matplotlib es la biblioteca estándar para crear gráficos en Python. Se importa como plt. (visualiza.py)

```
import numpy as np
import matplotlib.pyplot as plt

# Datos
dias = np.arange(1, 11) # Días del 1 al 10
temperaturas = np.array([28, 30, 25, 27, 29, 31, 26, 24, 32, 28])

# Crear una figura y un eje
plt.figure(figsize=(8, 5))

# Gráfico de líneas
plt.plot(dias, temperaturas, marker='o', linestyle='-', color='b',
label='Temperatura')

# Títulos y etiquetas
plt.title('Temperaturas en Cartago (últimos 10 días)')
plt.xlabel('Día')
plt.ylabel('Temperatura (°C)')
plt.grid(True) # Agregar una cuadrícula para mejor legibilidad
plt.legend()

# Mostrar el gráfico
#plt.show()

# En lugar de plt.show(), guardamos la imagen
plt.savefig('temperaturas.png', dpi=150, bbox_inches='tight')
print(" Gráfica guardada como 'temperaturas.png'")
```

Tipos de gráficos comunes en EDA:

- **Gráfico de líneas** (plt.plot): Para tendencias a lo largo del tiempo.
- **Gráfico de barras** (plt.bar): Para comparar categorías.
- **Histograma** (plt.hist): Para ver la distribución de una variable numérica.
- **Gráfico de dispersión** (plt.scatter): Para ver la relación entre dos variables numéricas.

Actividad en Clase

Objetivo: Cargar un archivo CSV con datos de cultivos, realizar un análisis exploratorio básico usando NumPy y generar visualizaciones con Matplotlib.

Preparación

1. Crear el archivo de datos: cultivos_detalle.csv

```
cultivo,hectareas,produccion_toneladas,region,tipo_suelo
Café,5,3.2,Centro,Franco
Caña,10,8.5,Norte,Arcilloso
Maíz,3,1.8,Sur,Franco
Frijol,4,2.1,Centro,Arcilloso
Arroz,6,4.0,Norte,Franco
Cacao,8,3.5,Sur,Franco
Palma,12,7.2,Centro,Arcilloso
Yuca,2,1.5,Norte,Franco
Plátano,7,5.0,Sur,Arcilloso
Tomate,1,0.8,Centro,Franco
```

2. Crear el script: eda_cultivos.py

Bloque 1: Importación de Librerías y Carga de Datos

```

"""
eda_cultivos.py
Script para realizar un Análisis Exploratorio de Datos (EDA) básico
sobre un archivo CSV de cultivos, utilizando NumPy y Matplotlib.
"""

import numpy as np
import matplotlib.pyplot as plt
import csv

def cargar_datos(archivo_csv):
    """
    Carga un archivo CSV y retorna un array de NumPy con los datos numéricos
    y una lista con los nombres de las columnas.
    """
    datos = []
    nombres_columnas = []
    try:
        with open(archivo_csv, 'r', encoding='utf-8') as archivo:
            lector = csv.reader(archivo)
            nombres_columnas = next(lector) # Leer la primera fila (encabezados)

            for fila in lector:
                # Convertir las columnas numéricas a float
                # Las columnas 1 (hectareas) y 2 (produccion) son numéricas
                hectareas = float(fila[1])
                produccion = float(fila[2])
                # Guardamos también el nombre del cultivo y la región para referencia
                datos.append([fila[0], hectareas, produccion, fila[3], fila[4]])

            print(f" Datos cargados correctamente: {len(datos)} registros.")
    except FileNotFoundError:
        print(f" Error: El archivo '{archivo_csv}' no existe.")
        return None, None
    except Exception as e:
        print(f" Error al leer el archivo: {e}")
        return None, None

    # Convertir a array de NumPy (solo las columnas numéricas para análisis)
    # Seleccionamos las columnas 1 (hectareas) y 2 (produccion)
    datos_numericos = np.array([[fila[1], fila[2]] for fila in datos], dtype=float)

    return datos_numericos, datos, nombres_columnas

```

Explicación Paso a Paso

1. Importación de librerías

```
import numpy as np
import matplotlib.pyplot as plt
import csv
```

- `import numpy as np`: Importa NumPy y le asigna el alias np. Es la convención estándar en la industria.
- `import matplotlib.pyplot as plt`: Importa el submódulo pyplot de Matplotlib, que contiene funciones para crear gráficos. plt es el alias estándar.
- `import csv`: Importa el módulo csv de la biblioteca estándar de Python, que nos permite leer y escribir archivos CSV.

Función `cargar_datos(archivo_csv)`

Esta función es la encargada de leer el archivo CSV y transformarlo en estructuras de datos que se puedan usar para el análisis.

Paso 1: Inicialización de variables

```
datos = []
nombres_columnas = []
```

- `datos`: Lista vacía donde almacenaremos cada fila del CSV como una lista.
- `nombres_columnas`: Lista vacía donde guardaremos los encabezados del CSV.

Paso 2: Manejo de excepciones (try-except)

```
try:
    with open(archivo_csv, 'r', encoding='utf-8') as archivo:
        # ... código para leer ...
except FileNotFoundError:
    print(f" Error: El archivo '{archivo_csv}' no existe.")
    return None, None
except Exception as e:
    print(f" Error al leer el archivo: {e}")
    return None, None
```

- try: Intenta ejecutar el código dentro del bloque.
- with open(...) as archivo: Abre el archivo en modo lectura ('r'). El with asegura que el archivo se cierre automáticamente al salir del bloque.
- encoding='utf-8': Especifica la codificación para evitar problemas con caracteres especiales (como tildes).
- except FileNotFoundError: Captura el error específico cuando el archivo no existe.
- except Exception as e: Captura cualquier otro error y muestra el mensaje.

Paso 3: Lectura del CSV con csv.reader

```
lector = csv.reader(archivo)
nombres_columnas = next(lector) # Leer la primera fila (encabezados)
```

- csv.reader(archivo): Crea un objeto lector que itera sobre las filas del CSV.
- next(lector): Obtiene la primera fila, que contiene los nombres de las columnas. La guardamos en nombres_columnas.

Paso 4: Procesamiento de cada fila

```
for fila in lector:
    hectareas = float(fila[1])
    produccion = float(fila[2])
    datos.append([fila[0], hectareas, produccion, fila[3], fila[4]])
```

- for fila in lector: Itera sobre cada fila restante del CSV.
- float(fila[1]): Convierte el segundo elemento de la fila (índice 1, que es "hectareas") a número decimal.
- float(fila[2]): Convierte el tercer elemento (índice 2, "produccion_toneladas") a número decimal.
- datos.append(...): Agrega una lista con los datos de la fila:
 - fila[0]: Nombre del cultivo (texto).
 - hectareas: Número decimal.
 - produccion: Número decimal.
 - fila[3]: Región (texto).
 - fila[4]: Tipo de suelo (texto)

Paso 5: Creación del array de NumPy

```
datos_numericos = np.array([[fila[1], fila[2]] for fila in datos], dtype=float)
```

- [[fila[1], fila[2]] for fila in datos]: Comprensión de lista que crea una nueva lista con solo las columnas numéricas (hectáreas y producción) de cada fila.
- np.array(..., dtype=float): Convierte esa lista en un array de NumPy, especificando que los datos son de tipo float.

Paso 6: Retorno de valores

```
return datos_numericos, datos, nombres_columnas
```

- Retorna:
 - datos_numericos: Array de NumPy con solo los datos numéricos.
 - datos: Lista completa con todos los datos (incluyendo texto).
 - nombres_columnas: Lista con los encabezados del CSV.

Bloque 2: Análisis Estadístico con NumPy

Código

```
def analizar_datos(datos_numericos):
    """Calcula estadísticas descriptivas de los datos numéricos."""
    if datos_numericos is None or len(datos_numericos) == 0:
        return None

    hectareas = datos_numericos[:, 0]
    produccion = datos_numericos[:, 1]

    estadisticas = {
        'hectareas': {
            'media': np.mean(hectareas),
            'mediana': np.median(hectareas),
            'desviacion': np.std(hectareas),
            'minimo': np.min(hectareas),
            'maximo': np.max(hectareas)
        },
        'produccion': {
            'media': np.mean(produccion),
            'mediana': np.median(produccion),
            'desviacion': np.std(produccion),
            'minimo': np.min(produccion),
            'maximo': np.max(produccion)
        }
    }
    return estadisticas
```

Explicación

Validación de entrada

```
if datos_numericos is None or len(datos_numericos) == 0:
    return None
```

- Verifica que `datos_numericos` no sea `None` y que tenga al menos un elemento. Si no, retorna `None` para evitar errores.

Extracción de columnas

```
hectareas = datos_numericos[:, 0]
produccion = datos_numericos[:, 1]
```

- `datos_numericos[:, 0]`: Selecciona **todas las filas** (:) y la **columna 0** (hectáreas). Esto crea un array 1D con todos los valores de hectáreas.
- `datos_numericos[:, 1]`: Selecciona todas las filas y la columna 1 (producción).

Cálculo de estadísticas

```
estadisticas = {  
    'hectareas': {  
        'media': np.mean(hectareas),  
        'mediana': np.median(hectareas),  
        'desviacion': np.std(hectareas),  
        'minimo': np.min(hectareas),  
        'maximo': np.max(hectareas)  
    },  
    'produccion': {  
        'media': np.mean(produccion),  
        'mediana': np.median(produccion),  
        'desviacion': np.std(produccion),  
        'minimo': np.min(produccion),  
        'maximo': np.max(produccion)  
    }  
}
```

- `np.mean()`: Calcula el promedio (media aritmética).
- `np.median()`: Calcula la mediana (valor central).
- `np.std()`: Calcula la desviación estándar (dispersión de los datos).
- `np.min()`: Encuentra el valor mínimo.
- `np.max()`: Encuentra el valor máximo.

Todas estas funciones son **vectorizadas**, lo que significa que operan sobre todo el array de forma eficiente, sin necesidad de bucles.

Retorno del diccionario

```
return estadísticas
```

- Retorna un diccionario anidado con todas las estadísticas calculadas, organizadas por variable.

Bloque 3: Visualización con Matplotlib

Código

```
def generar_visualizaciones(datos_numericos, datos_completos):
    """Genera gráficos para el análisis exploratorio."""
    if datos_numericos is None or len(datos_numericos) == 0:
        return

    hectareas = datos_numericos[:, 0]
    produccion = datos_numericos[:, 1]
    nombres = [fila[0] for fila in datos_completos] # Nombres de los cultivos

    # 1. Gráfico de barras: Hectáreas por cultivo
    plt.figure(figsize=(10, 5))
    plt.bar(nombres, hectareas, color='skyblue')
    plt.title('Hectáreas por Cultivo')
    plt.xlabel('Cultivo')
    plt.ylabel('Hectáreas')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.savefig('hectareas_por_cultivo.png') # Guardar la imagen
    plt.show()

    # 2. Gráfico de dispersión: Hectáreas vs Producción
    plt.figure(figsize=(8, 6))
    plt.scatter(hectareas, produccion, color='green', alpha=0.7)
    plt.title('Relación: Hectáreas vs Producción (toneladas)')
    plt.xlabel('Hectáreas')
    plt.ylabel('Producción (toneladas)')
    plt.grid(True)
    plt.savefig('hectareas_vs_produccion.png')
    plt.show()

    # 3. Histograma: Distribución de la producción
    plt.figure(figsize=(8, 5))
    plt.hist(produccion, bins=5, color='orange', edgecolor='black')
    plt.title('Distribución de la Producción')
    plt.xlabel('Producción (toneladas)')
    plt.ylabel('Frecuencia')
    plt.grid(True, axis='y', linestyle='--', alpha=0.7)
    plt.savefig('distribucion_produccion.png')
    plt.show()
```

Explicación

1. Validación de entrada y extracción de datos

```
if datos_numericos is None or len(datos_numericos) == 0:
    return

hectareas = datos_numericos[:, 0]
produccion = datos_numericos[:, 1]
nombres = [fila[0] for fila in datos_completos]
```

- Verifica que los datos no estén vacíos.
- Extrae las columnas numéricas.
- Crea una lista nombres con el nombre de cada cultivo (columna 0 de datos_completos).

Gráfico de barras

```
plt.figure(figsize=(10, 5))
plt.bar(nombres, hectareas, color='skyblue')
plt.title('Hectáreas por Cultivo')
plt.xlabel('Cultivo')
plt.ylabel('Hectáreas')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('hectareas_por_cultivo.png')
plt.show()
```

- `plt.figure(figsize=(10, 5))`: Crea una figura de 10 pulgadas de ancho y 5 de alto.
- `plt.bar(nombres, hectareas, color='skyblue')`: Gráfico de barras. El eje X son los nombres de los cultivos, el eje Y son las hectáreas.
- `plt.title()`, `plt.xlabel()`, `plt.ylabel()`: Agregan títulos y etiquetas.
- `plt.xticks(rotation=45, ha='right')`: Rota las etiquetas del eje X 45 grados para que no se superpongan y las alinea a la derecha.
- `plt.tight_layout()`: Ajusta automáticamente los márgenes para que todo quepa.
- `plt.savefig('hectareas_por_cultivo.png')`: Guarda la imagen en un archivo PNG.
- `plt.show()`: Muestra la gráfica en pantalla (si hay display disponible).

Gráfico de dispersión

```
plt.figure(figsize=(8, 6))
plt.scatter(hectareas, produccion, color='green', alpha=0.7)
plt.title('Relación: Hectáreas vs Producción (toneladas)')
plt.xlabel('Hectáreas')
plt.ylabel('Producción (toneladas)')
plt.grid(True)
plt.savefig('hectareas_vs_produccion.png')
plt.show()
```

- `plt.scatter(x, y, color, alpha)`: Crea un gráfico de dispersión.
- `alpha=0.7`: Hace los puntos ligeramente transparentes para ver superposiciones.
- `plt.grid(True)`: Agrega una cuadrícula para facilitar la lectura.

Histograma

```
plt.figure(figsize=(8, 5))
plt.hist(produccion, bins=5, color='orange', edgecolor='black')
plt.title('Distribución de la Producción')
plt.xlabel('Producción (toneladas)')
plt.ylabel('Frecuencia')
plt.grid(True, axis='y', linestyle='--', alpha=0.7)
plt.savefig('distribucion_produccion.png')
plt.show()
```

- `plt.hist(datos, bins, color, edgecolor)`: Crea un histograma.
- `bins=5`: Divide los datos en 5 intervalos (barras).
- `edgecolor='black'`: Borde negro en cada barra para mejor visibilidad.
- `plt.grid(True, axis='y', linestyle='--', alpha=0.7)`: Cuadrícula solo en el eje Y, con líneas punteadas y transparencia.

Bloque 4: Integración y Ejecución del Flujo Completo

Código

```
def main():
    """Función principal del programa."""
    print("=" * 50)
    print(" ANÁLISIS EXPLORATORIO DE DATOS (EDA) - CULTIVOS")
    print("=" * 50)

    # Cargar datos
    datos_numericos, datos_completos, columnas = cargar_datos('cultivos_detalle.csv')
    if datos_numericos is None:
        return

    # Analizar datos
    estadisticas = analizar_datos(datos_numericos)
    if estadisticas:
        print("\n ESTADÍSTICAS DESCRIPTIVAS:")
        print("-" * 40)
        for variable, valores in estadisticas.items():
            print(f"\n{variable.upper()}:")
            for key, value in valores.items():
                print(f" {key.capitalize()}: {value:.2f}")

    # Generar visualizaciones
    print("\n GENERANDO VISUALIZACIONES...")
    generar_visualizaciones(datos_numericos, datos_completos)
    print("Visualizaciones guardadas como archivos PNG.")

    print("\n Análisis completado exitosamente.")

if __name__ == "__main__":
    main()
```

Explicación Paso a Paso

Función main()

Es la función principal que orquesta todo el flujo del programa.

Carga de datos

```
datos_numericos, datos_completos, columnas = cargar_datos('cultivos_detalle.csv')
if datos_numericos is None:
    return
```

- Llama a `cargar_datos()` con el nombre del archivo.
- Si retorna `None` (por error), el programa termina con `return`.

Análisis estadístico

```
estadisticas = analizar_datos(datos_numericos)
if estadisticas:
    print("\n ESTADÍSTICAS DESCRIPTIVAS:")
    print("-" * 40)
    for variable, valores in estadisticas.items():
        print(f"\n{variable.upper()}:")
        for key, value in valores.items():
            print(f"    {key.capitalize()}: {value:.2f}")
```

- Llama a `analizar_datos()` y guarda el resultado.
- Si hay estadísticas, las imprime de forma legible:
 - `for variable, valores in estadisticas.items():` Itera sobre cada variable (hectáreas, producción).
 - `for key, value in valores.items():` Itera sobre cada estadística (media, mediana, etc.).
 - `f"{key.capitalize()}: {value:.2f}"`: Formatea el texto con la primera letra mayúscula y el valor con 2 decimales.

Visualización

```
print("\n GENERANDO VISUALIZACIONES...")
generar_visualizaciones(datos_numericos, datos_completos)
print("Visualizaciones guardadas como archivos PNG.")
```

- Llama a `generar_visualizaciones()` con los datos.
- Muestra mensajes de progreso.

Bloque `if __name__ == "__main__":`

```
if __name__ == "__main__":
    main()
```

- Esta línea asegura que `main()` solo se ejecute si el script se ejecuta directamente (no si se importa como módulo en otro script).

Resumen de la División del Código

Bloque	Archivo	Función Principal	Lo que Aprenden
1	eda_cultivos.py (parte 1)	cargar_datos()	Importación de librerías, manejo de archivos CSV, creación de arrays NumPy.
2	eda_cultivos.py (parte 2)	analizar_datos()	Estadísticas descriptivas vectorizadas con NumPy.
3	eda_cultivos.py (parte 3)	generar_visualizaciones()	Creación de gráficos con Matplotlib (barras, dispersión, histograma).
4	eda_cultivos.py (completo)	main()	Integración de todo el flujo, formato de salida y ejecución completa.

Actividad Práctica para el Bloque 4

1. Completar el script con la función `main()` y el bloque `if __name__ == "__main__":`.
2. Ejecutar el script completo.
3. Verificar la salida en consola y los archivos PNG generados.
4. Agregar una nueva funcionalidad: guardar las estadísticas en un archivo `estadisticas.txt` usando `json.dump()`.

Versión Final del Código Completo (con savefig y sin plt.show())

```

"""
eda_cultivos.py
Script para realizar un Análisis Exploratorio de Datos (EDA) básico
sobre un archivo CSV de cultivos, utilizando NumPy y Matplotlib.
"""

import numpy as np
import matplotlib.pyplot as plt
import csv

def cargar_datos(archivo_csv):
    """
    Carga un archivo CSV y retorna un array de NumPy con los datos numéricos
    y una lista con los nombres de las columnas.
    """
    datos = []
    nombres_columnas = []
    try:
        with open(archivo_csv, 'r', encoding='utf-8') as archivo:
            lector = csv.reader(archivo)
            nombres_columnas = next(lector) # Leer la primera fila (encabezados)

            for fila in lector:
                # Convertir las columnas numéricas a float
                # Las columnas 1 (hectareas) y 2 (produccion) son numéricas
                hectareas = float(fila[1])
                produccion = float(fila[2])
                # Guardamos también el nombre del cultivo y la región para
                referencia
                datos.append([fila[0], hectareas, produccion, fila[3], fila[4]])

            print(f" Datos cargados correctamente: {len(datos)} registros.")
    except FileNotFoundError:
        print(f" Error: El archivo '{archivo_csv}' no existe.")
        return None, None
    except Exception as e:
        print(f" Error al leer el archivo: {e}")
        return None, None

    # Convertir a array de NumPy (solo las columnas numéricas para análisis)
    # Seleccionamos las columnas 1 (hectareas) y 2 (produccion)
    datos_numericos = np.array([[fila[1], fila[2]] for fila in datos], dtype=float)

    return datos_numericos, datos, nombres_columnas

def analizar_datos(datos_numericos):
    """Calcula estadísticas descriptivas de los datos numéricos."""
    if datos_numericos is None or len(datos_numericos) == 0:
        return None

    hectareas = datos_numericos[:, 0]
    produccion = datos_numericos[:, 1]

    estadisticas = {
        'hectareas': {
            'media': np.mean(hectareas),
            'mediana': np.median(hectareas),

```

```

        'desviacion': np.std(hectareas),
        'minimo': np.min(hectareas),
        'maximo': np.max(hectareas)
    },
    'produccion': {
        'media': np.mean(produccion),
        'mediana': np.median(produccion),
        'desviacion': np.std(produccion),
        'minimo': np.min(produccion),
        'maximo': np.max(produccion)
    }
}
return estadisticas

def generar_visualizaciones(datos_numericos, datos_completos):
    """Genera gráficos para el análisis exploratorio."""
    if datos_numericos is None or len(datos_numericos) == 0:
        return

    hectareas = datos_numericos[:, 0]
    produccion = datos_numericos[:, 1]
    nombres = [fila[0] for fila in datos_completos] # Nombres de los cultivos

    # 1. Gráfico de barras: Hectáreas por cultivo
    plt.figure(figsize=(10, 5))
    plt.bar(nombres, hectareas, color='skyblue')
    plt.title('Hectáreas por Cultivo')
    plt.xlabel('Cultivo')
    plt.ylabel('Hectáreas')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.savefig('hectareas_por_cultivo.png') # Guardar la imagen
    plt.show()

    # 2. Gráfico de dispersión: Hectáreas vs Producción
    plt.figure(figsize=(8, 6))
    plt.scatter(hectareas, produccion, color='green', alpha=0.7)
    plt.title('Relación: Hectáreas vs Producción (toneladas)')
    plt.xlabel('Hectáreas')
    plt.ylabel('Producción (toneladas)')
    plt.grid(True)
    plt.savefig('hectareas_vs_produccion.png')
    plt.show()

    # 3. Histograma: Distribución de la producción
    plt.figure(figsize=(8, 5))
    plt.hist(produccion, bins=5, color='orange', edgecolor='black')
    plt.title('Distribución de la Producción')
    plt.xlabel('Producción (toneladas)')
    plt.ylabel('Frecuencia')
    plt.grid(True, axis='y', linestyle='--', alpha=0.7)
    plt.savefig('distribucion_produccion.png')
    plt.show()

def main():
    """Función principal del programa."""
    print("=" * 50)
    print(" ANÁLISIS EXPLORATORIO DE DATOS (EDA) - CULTIVOS")

```

```

print("=" * 50)

# Cargar datos
datos_numericos, datos_completos, columnas =
cargar_datos('cultivos_detalle.csv')
if datos_numericos is None:
    return

# Analizar datos
estadisticas = analizar_datos(datos_numericos)
if estadisticas:
    print("\n ESTADÍSTICAS DESCRIPTIVAS:")
    print("-" * 40)
    for variable, valores in estadisticas.items():
        print(f"\n{variable.upper()}:")
        for key, value in valores.items():
            print(f"  {key.capitalize()}: {value:.2f}")

# Generar visualizaciones
print("\n GENERANDO VISUALIZACIONES...")
generar_visualizaciones(datos_numericos, datos_completos)
print("Visualizaciones guardadas como archivos PNG.")

print("\n Análisis completado exitosamente.")

if __name__ == "__main__":
    main()

```

Ejecutar el Script

Desde la terminal de WSL (con el entorno virtual activado):

```
python eda_cultivos.py
```

Interpretación de los Resultados

1. **Estadísticas descriptivas:** deben observar los valores de media, mediana, desviación estándar, etc., para entender la tendencia central y la dispersión de los datos.
2. **Gráfico de barras:** Permite comparar visualmente las hectáreas de cada cultivo. ¿Cuál tiene más? ¿Cuál tiene menos?
3. **Gráfico de dispersión:** Muestra la relación entre hectáreas y producción. ¿A mayores hectáreas, mayor producción? ¿Hay algún cultivo que destaque (un punto atípico)?
4. **Histograma:** Muestra cómo se distribuye la producción. ¿Es una distribución simétrica? ¿Hay muchos cultivos con producción baja o alta?

Actividad Independiente (Complementaria)

Objetivo: Aplicar los conceptos de NumPy y EDA al dataset del proyecto personal.

Instrucciones detalladas

1. **Tomar el archivo de datos** (CSV) que seleccionaron para su proyecto.
2. **Crear un script** `eda_proyecto.py` que:
 - Cargue el archivo CSV usando `csv.DictReader` o `np.genfromtxt`.
 - Convierta las columnas numéricas relevantes en un array de NumPy.
 - Calcule las siguientes estadísticas para al menos **dos** variables numéricas:
 - Media (`np.mean`)
 - Mediana (`np.median`)
 - Desviación estándar (`np.std`)
 - Mínimo y máximo (`np.min`, `np.max`)
 - Genere al menos **dos** gráficos con Matplotlib:
 - Un histograma de una variable numérica.
 - Un gráfico de dispersión que muestre la relación entre dos variables numéricas.
 - Guarde los gráficos como archivos PNG.
3. **Documentar los hallazgos:**
 - En el archivo `README.md` del proyecto, agregar una sección "Análisis Exploratorio de Datos" donde describan:
 - ¿Qué variables analizaron?
 - ¿Qué estadísticas encontraron?
 - ¿Qué patrones o relaciones observaron en los gráficos?
 - ¿Cómo estos hallazgos pueden influir en el modelo de IA que planean construir?
4. **Subir los cambios a GitHub** con un commit descriptivo.

Cierre y Preguntas

Resumen de la clase

- **NumPy:**
 - Es la base del cómputo numérico en Python.
 - Los arrays son más eficientes que las listas.
 - Las operaciones vectorizadas son la clave de su rendimiento.
 - El *broadcasting* permite operar con arrays de diferentes formas.
- **EDA (Análisis Exploratorio de Datos):**
 - Es un paso fundamental antes de modelar.
 - Nos ayuda a entender la estructura, distribución y relaciones en los datos.
 - Las estadísticas descriptivas y las visualizaciones son sus herramientas principales.

Conexión con el Proyecto

El EDA que se realizó en la clase es el mismo proceso que deben seguir con sus propios datos. Cuanto mejor entiendan sus datos, mejor podrán construir su modelo de IA.

Próxima clase

- Parcial: 14 de septiembre
- Sustentación de proyectos: 21 de septiembre
- Clase 28 de septiembre:
 - **Pandas:** La biblioteca definitiva para manipulación y análisis de datos.
 - **Visualización avanzada con Seaborn.**
 - **Preparación de datos para machine learning.**

Anexo: Cheat Sheet de NumPy y Matplotlib

Concepto	Sintaxis	Ejemplo
Importar NumPy	<code>import numpy as np</code>	
Crear array	<code>np.array(lista)</code>	<code>np.array([1, 2, 3])</code>
Array de ceros	<code>np.zeros(shape)</code>	<code>np.zeros((2, 3))</code>
Array de unos	<code>np.ones(shape)</code>	<code>np.ones((3, 2))</code>
Secuencia	<code>np.arange(start, stop, step)</code>	<code>np.arange(0, 10, 2)</code>
Puntos equidistantes	<code>np.linspace(start, stop, num)</code>	<code>np.linspace(0, 1, 5)</code>
Números aleatorios	<code>np.random.rand(shape)</code>	<code>np.random.rand(2, 3)</code>
Shape	<code>array.shape</code>	<code>matriz.shape</code>
Media	<code>np.mean(array)</code>	<code>np.mean(temperaturas)</code>
Mediana	<code>np.median(array)</code>	<code>np.median(temperaturas)</code>
Desviación estándar	<code>np.std(array)</code>	<code>np.std(temperaturas)</code>
Máximo	<code>np.max(array)</code>	<code>np.max(temperaturas)</code>
Mínimo	<code>np.min(array)</code>	<code>np.min(temperaturas)</code>
Suma	<code>np.sum(array)</code>	<code>np.sum(hectareas)</code>
Indexación	<code>array[fila, columna]</code>	<code>matriz[1, 2]</code>
Slicing	<code>array[inicio:fin:paso]</code>	<code>arr[1:4]</code>
Broadcasting	Operación entre arrays de diferente forma	<code>a + b</code>
Importar Matplotlib	<code>import matplotlib.pyplot as plt</code>	
Gráfico de líneas	<code>plt.plot(x, y)</code>	<code>plt.plot(dias, temp)</code>
Gráfico de barras	<code>plt.bar(x, y)</code>	<code>plt.bar(cultivos, hectareas)</code>

Histograma	<code>plt.hist(data)</code>	<code>plt.hist(produccion)</code>
Gráfico de dispersión	<code>plt.scatter(x, y)</code>	<code>plt.scatter(hectareas, produccion)</code>
Título	<code>plt.title('texto')</code>	
Etiqueta eje X	<code>plt.xlabel('texto')</code>	
Etiqueta eje Y	<code>plt.ylabel('texto')</code>	
Cuadrícula	<code>plt.grid(True)</code>	
Guardar figura	<code>plt.savefig('nombre.png')</code>	
Mostrar figura	<code>plt.show()</code>	